

```

// PowerPlayAutonomous - Nikhil
// simplified take on POWER PLAY, the 22/23 ftc season game. robots picked up
// cones and placed them onto "junctions" - poles at 4 different heights (ground,
// low, medium, high), worth more points the taller they are. there's also a
// randomised "signal cone" read during auto that tells you which zone to park in
// at the end of autonomous.
// this version uses the distance sensor to represent driving up to a junction
// and "placing" a cone (again just standing in with a short pause since i don't
// have a claw/lift mechanism built), then uses a colour reading to decide which
// of three parking zones to head to at the end, mimicking the randomised signal
// cone logic from the real game

public class PowerPlayAutonomous extends LinearOpMode {
    DcMotor motorLeft;
    DcMotor motorRight;
    ColorSensor color1;
    DistanceSensor distance1;

    @Override
    public void runOpMode() {
        motorLeft = hardwareMap.get(DcMotor.class, "motorLeft");
        motorRight = hardwareMap.get(DcMotor.class, "motorRight");
        color1 = hardwareMap.get(ColorSensor.class, "color1");
        distance1 = hardwareMap.get(DistanceSensor.class, "distance1");

        motorLeft.setDirection(DcMotor.Direction.REVERSE);
        motorLeft.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
        motorRight.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);

        waitForStart();

        // drive up to the junction, stopping a set distance away rather than
        // guessing a time - same pattern as my colour detection file
        while (opModeIsActive() && distance1.getDistance(DistanceUnit.CM) > 20) {
            motorLeft.setPower(0.5);
            motorRight.setPower(0.5);
            telemetry.addData("Distance to junction", distance1.getDistance(DistanceUnit.CM));
            telemetry.update();
        }
        motorLeft.setPower(0);
        motorRight.setPower(0);

        // "place the cone" - standing in for an actual scoring mechanism
        sleep(300);

        // read the "signal cone" and head to the matching parking zone,
        // same three way branch idea as freight frenzy's barcode read, just
        // used here for endgame parking instead of hub level choice
        motorLeft.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
        motorRight.setMode(DcMotor.RunMode.RUN_USING_ENCODER);

        telemetry.addData("Red Value", color1.red());
        telemetry.addData("Blue Value", color1.blue());
        telemetry.update();

        if (color1.red() > 220) {
            moveTo(-800, -800, 0.5); // zone 1
        } else if (color1.blue() > 220) {
            moveTo(0, 0, 0.5); // zone 2 - stay put, already centred
        } else {
            moveTo(800, 800, 0.5); // zone 3
        }
        motorLeft.setPower(0);
    }
}

```

```

        motorRight.setPower(0);
    }

    private void moveTo(int leftTicks, int rightTicks, double power) {
        int leftTarget = motorLeft.getCurrentPosition() + leftTicks;
        int rightTarget = motorRight.getCurrentPosition() + rightTicks;

        motorLeft.setTargetPosition(leftTarget);
        motorRight.setTargetPosition(rightTarget);

        motorLeft.setMode(DcMotor.RunMode.RUN_TO_POSITION);
        motorRight.setMode(DcMotor.RunMode.RUN_TO_POSITION);

        motorLeft.setPower(power);
        motorRight.setPower(power);

        while (opModeIsActive() && motorLeft.isBusy() && motorRight.isBusy()) {
            telemetry.addData("Left ticks", motorLeft.getCurrentPosition());
            telemetry.addData("Right ticks", motorRight.getCurrentPosition());
            telemetry.update();
        }

        motorLeft.setPower(0);
        motorRight.setPower(0);
    }
}

```